

Arquitetura de Computadores I

Licenciatura em Engenharia Informática

Miguel Barão

Ano Letivo 2025–2026

Resumo

Pretende-se implementar um programa que interprete uma mini-linguagem de gráficos vetoriais e gere imagens *raster* correspondentes. O programa deverá ler um ficheiro com uma sequência de primitivas gráficas e produzir um ficheiro de texto ASCII com uma imagem.

1 Imagens *raster* e vetoriais

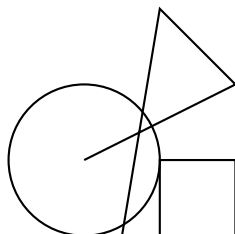
Existem dois tipos comuns de representação de imagens: *raster* e *vetorial*.

As imagens *raster* são simplesmente grelhas de píxeis. A resolução de uma imagem é determinada pela dimensão da grelha de píxeis, que é fixa, pelo que a qualidade piora ao ampliar uma região da imagem (o número de píxeis é menor). As imagens deste tipo são normalmente usadas em fotografia, onde a captura de uma imagem com uma câmara produz diretamente uma grelha de píxeis. Exemplos de formatos de imagens raster são o PNG, BMP, JPEG, TIFF, PBM. Uma imagem em tons de cinzento é representada por uma matriz em que os elementos são as intensidades dos píxeis:

$$\begin{bmatrix} I_{11} & I_{12} & \cdots & I_{1n} \\ I_{21} & I_{22} & \cdots & I_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ I_{m1} & I_{m2} & \cdots & I_{mn} \end{bmatrix}.$$

No caso de imagens a cores, a cor de cada píxel é representada nas componentes de cor RGB (*Red*, *Green*, *Blue*).

As imagens *vetoriais* usam linhas, pontos, curvas e polígonos para representar imagens em vez de píxeis:



Nesta representação, em vez de usar uma grelha de píxeis, são usadas coordenadas, comprimentos e outros parâmetros para representar as figuras geométricas. Este tipo de

imagens é usado nos formatos SVG (Scalable Vector Graphics) e em aplicações gráficas de ilustração. Os gráficos produzidos podem ser ampliados arbitrariamente, sem perda de qualidade, uma vez que as coordenadas e parâmetros que definem as figuras não mudam.

2 Formato MyVG

O formato de imagens vetoriais **MyVG** usa uma linguagem desenvolvida especificamente para este trabalho. O formato consiste simplesmente em texto ASCII com primitivas gráficas executadas em sequência para produzir uma imagem final.

As primitivas permitem definir figuras geométricas tais como linhas, pontos e polígonos:

```
<largura> <altura>      # dimensao da imagem (largura, altura)
p <x> <y>                 # ponto nas coordenadas (x, y)
r <x1> <y1> <x2> <y2>    # retangulo com vertices em (x1, y1) e (x2, y2)
l <x1> <y1> <x2> <y2>    # linha reta entre (x1, y1) e (x2, y2)
```

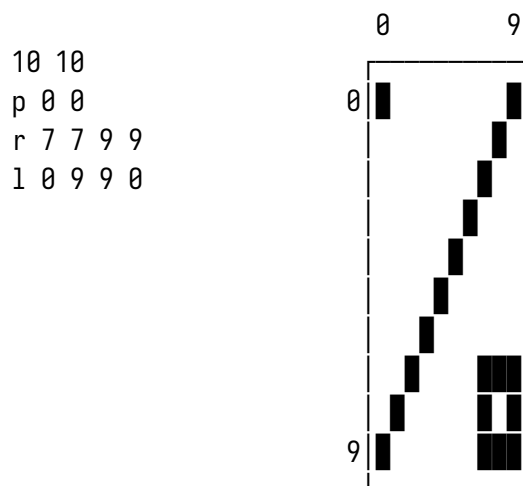
A primeira linha do ficheiro define a dimensão da imagem final em píxeis. A possibilidade de ter comentários nos ficheiros **MyVG** é opcional.

3 Funcionalidades a implementar

A implementação do trabalho é composta por um conjunto obrigatório de funcionalidades para obter uma nota mínima e um conjunto de funcionalidades adicionais que somam à nota mínima até ao valor máximo de 20 valores.

As funcionalidades obrigatórias são: Pedir ao utilizador o nome de um ficheiro **MyVG**; Interpretar as primitivas **p**, **r** e **l** descritas na secção 2 produzindo o resultado num array que contém a imagem; Guardar a imagem resultante num ficheiro ASCII.

Por exemplo, o ficheiro **MyVG** da esquerda produz um ficheiro com a imagem mostrada à direita:



No ficheiro ASCII com a imagem gerada, os píxeis brancos devem ser representados por um espaço e os pretos pelo carácter **#**. A imagem acima mostra blocos pretos no lugar do cardinal apenas para efeitos de ilustração.

Para além das funcionalidades obrigatórias descritas anteriormente, as funcionalidades adicionais são as seguintes:

```
# permitir comentarios
z <x1> <y1> ... <xn> <yn> # poligono com os vertices indicados
f <x> <y> <ficheiro>      # copia conteudo de um ficheiro com uma imagem
                           # para as coordenadas (x,y)
```

O polígono (z) permite desenhar triângulos, pentágonos, hexágonos, *etc.* A implementação do polígono pode ser realizada chamando várias vezes a função que desenha linhas retas.

A inclusão de um ficheiro (f) consiste em copiar uma imagem previamente gerada pelo programa, e que já se encontra guardada num ficheiro, para as coordenadas indicadas de modo a que estas coordenadas correspondam ao canto superior esquerdo da imagem.

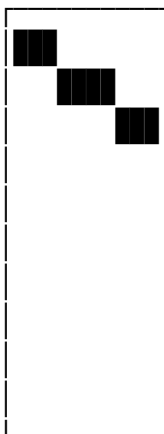
A deteção e tratamento de situações de erro, como por exemplo usar coordenadas fora da dimensão da imagem, fica ao critério dos alunos (ignorar ou reportar o erro).

Todos os cálculos devem ser realizados em aritmética inteira (sem vírgula flutuante). São valorizados os trabalhos que não usem as instruções de multiplicação, divisão ou resto.

4 Algoritmo para desenhar linhas

A equação da reta, $y = mx + b$, permite colocar pontos nas coordenadas (x, y) dados o declive m da reta e a interseção b com o eixo das ordenadas. Seria em princípio possível desenhar uma reta fazendo a variável x varrer todos os pontos entre x_1 e x_2 e calculando os valores de y correspondentes. Esta abordagem requer a utilização de números fracionários, operações de multiplicação/divisão, e conversão dos números fracionários para inteiros de modo a obter coordenadas (x, y) inteiras.

A utilização direta da equação da reta é desadequada, tanto pela necessidade de realizar multiplicações e divisões como pelo facto de se ter de lidar com números fracionários. Em alternativa pode usar-se um algoritmo que funciona exclusivamente com números inteiros e que não requer operações de multiplicação nem divisão. Para ter uma ideia intuitiva sobre como funciona o algoritmo, considere a imagem seguinte onde o declive da reta satisfaz $0 \leq m \leq 1$ (note que o eixo y tem origem no canto superior esquerdo e cresce para baixo):



Quando o declive é inferior a 1, é necessário percorrer todos os valores de x na horizontal de 1 em 1, enquanto que os valores de y ou se mantêm ou incrementam em 1 valor. Assim,

pode fazer-se um ciclo em que em cada passo só temos de decidir se y incrementa ou não. Com esta abordagem deixa de ser necessário usar multiplicações e divisões.

Nas situações em que o declive é superior a 1, o raciocínio é o mesmo com a diferença de que se percorrem todos os valores de y e em cada passo decide-se se se incrementa x .

Os incrementos em x ou em y podem ser negativos se o declive da reta for negativo.

A descrição completa do algoritmo é a que se mostra no diagrama da figura 1.

5 Desenvolvimento do trabalho

O trabalho deve ser desenvolvido em duas fases. Na primeira fase é realizada uma implementação das funcionalidades obrigatórias na linguagem C. Esta parte do trabalho destina-se apenas à estruturação do trabalho em funções e não será objeto de avaliação. A segunda fase do trabalho, consiste na implementação em Assembly do programa realizado em C. Este trabalho está sujeito a avaliação no final. Ambas as fases devem ser submetidas no moodle.

No início do código deve estar a seguinte declaração: “Este trabalho foi integralmente realizado pelos alunos X e Y sem recurso a ferramentas de inteligência artificial”.

6 Submissão dos trabalhos

- Os trabalhos devem ser realizados em **grupos de 2 alunos**.
- Cada grupo deve submeter, no moodle, um único ficheiro comprimido contendo o código Assembly e um relatório em PDF. Apenas um dos alunos do grupo submete o trabalho.
- O nome do ficheiro comprimido deve ser formado pelos números dos alunos por ordem crescente, por exemplo 11111-22222.zip.
- Não é permitida a utilização de ferramentas que automatizem a escrita de código, tais como compiladores ou ferramentas de inteligência artificial como o chatGPT, Gemini, DeepSeek, Claude, *etc*.
- Não é permitido partilhar código com outros grupos ou usar código já existente.
- Qualquer situação de fraude implica a reprovação à unidade curricular e será obrigatoriamente reportada para instauração de procedimento disciplinar, conforme estipulado no regulamento académico.

Bom trabalho! 👍
Miguel Barão

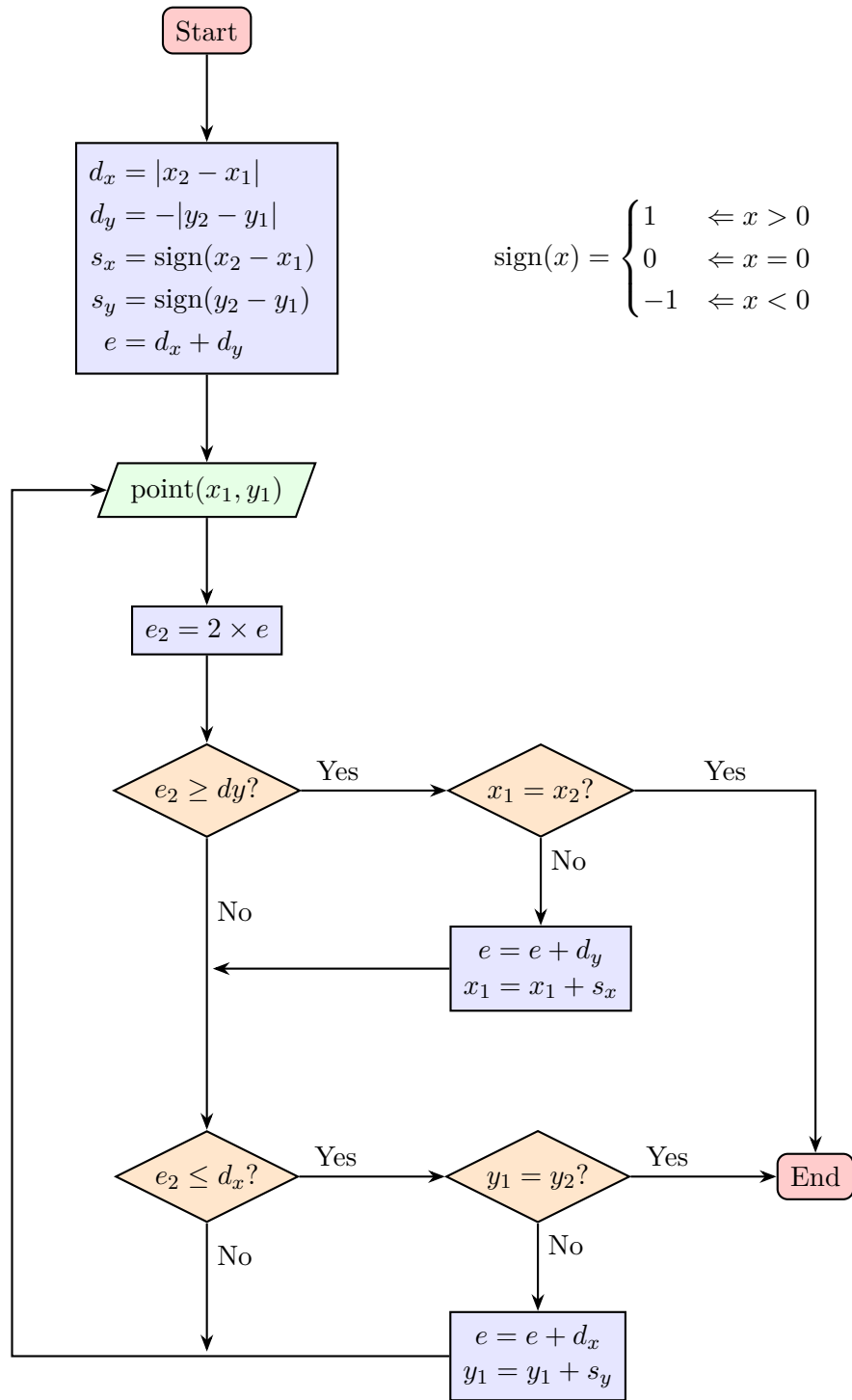


Figura 1: Algoritmo para traçar uma reta entre dois pontos.